

Assignment on Sed and Awk

Release date: 23rd May

Submission date: 2nd June

In this age of Python and R, should one care about `sed` and `awk`? In fact, there are good reasons why one should.

1. These tools remain valuable because they work directly in the **Unix text world** where much system data already lives: log files in `/var/log`, outputs of tools like `ps`, `ls`, `git`, `nm`, `readelf`, `objdump` and text flowing through pipelines. Apart from being available in the local machine, this plaintext information could be also be accessed from remote machines (through `ssh`).
2. `sed` and `awk` are especially well suited to streaming data. They process input **line by line, often in constant memory**, and fit naturally into computational pipelines. For example, in a very large CSV-like file, we can filter and aggregate in one pass:

```
awk -F, '$3 > 500 {sum[$9]+=$3} END {for (s in sum) print s, sum[s]}' big.csv
```

3. These tools are **lightweight and fast**. For many tasks, they can process large files in seconds or minutes, with very little setup.
4. When Python is better:
 - a. when the logic is complex
 - b. real data structures are needed
 - c. testing and maintainability matter
 - d. libraries, APIs, JSON, HTTP, or plotting are involved

In other words, when we want an actual software instead of helper.

1. This assignment is meant to illustrate three strengths of tools like `sed` and `awk`: (i) they work where the data already is, (ii) they are excellent for quick investigation, and (iii) they are extremely effective for one-shot transformations.

Assume that the provided tarball has been extracted, creating a top-level directory named `q1_inputs`. The relevant directory structure is shown below.

```
./q1_inputs/  
├── logs/  
│   ├── auth.log  
│   └── app.log
```

```

└─ src/
   ├── README.txt
   ├── include/
   │   ├── core/
   │   │   ├── adapter.h
   │   │   └── legacy_wrap.h
   │   ├── moduleA/
   │   │   └── http_client.h
   │   └── moduleB/
   │       └── metrics.h
   ├── legacy/
   │   └── net/
   │       ├── adapter_boot.c
   │       ├── adapter_reset.h
   │       └── connection.c
   ├── module1/
   │   └── io/
   │       ├── file_reader.c
   │       ├── init_helpers.h
   │       ├── init_sequence.c
   │       └── parser.c
   └── module2/
       ├── drivers/
       │   ├── serial/
       │   │   ├── serial_boot.c
       │   │   ├── serial_diag.c
       │   │   ├── serial_diag.h
       │   │   └── subsys/
       │   │       ├── recovery.c
       │   │       └── recovery.h
       │   └── usb/
       │       ├── adapter_probe.c
       │       ├── adapter_probe.h
       │       └── tempdebug_notes.c
       └── tools/
           └── report.awk

```

All solutions must work on this directory tree. In particular, Part B must not assume that the relevant `*.c` and `*.h` files live at a fixed depth.

You may use `bash`, `sed`, `awk`, and standard Unix utilities. *Do not use Python, R, Perl, spreadsheets, or GUI tools.* The questions `awk-a/b/c` must use **awk**, and `sed-a/b` must use **sed**.

awk-1. Write a shell script called `failed_login.sh` with usage

`./failed_login.sh <input> <output>`

which when given a file `./q1_inputs/logs/auth.log` as **<input>** will create a file `<output>/failed_logins.txt` that will extract failed login attempts: Each line of this file must contain the following fields, in this order: **month, day, time, username, and source IP address.**

Your extraction must correctly handle both of the following forms that occur in the log:

```
Jul 10 08:11:09 labnode42 sshd[1800]: Failed password
for root from 10.5.12.77 port 51322 ssh2
Jul 10 08:11:09 labnode42 sshd[1900]: Failed password
for invalid user guest from 185.199.108.7 port 59201
ssh2
```

In other words, your solution must extract `root` in the first case and `guest` in the second case. **[5 marks]**

awk-2. Write a script called `summary.sh` with usage

```
./summary.sh <input> <output>
```

that summarizes the list of failed users, i.e. given `./q1_inputs/logs/auth.log` as `<input>`, will create a file `<output>/top_failed_users.txt` containing the top 10 usernames by number of failed login attempts, sorted in descending order of count. If two usernames have the same count, sort them in ascending lexicographic order of usernames.

The solution must use `awk` both to extract selected fields from `auth.log` and to compute the username-frequency summary. **[5 marks]**

awk-3. Now write a script `local_filter.sh` with usage

```
local_filter.sh <input> <output>
```

that, given a input `./q1_inputs/logs/app.log` which will filter lines containing `ERROR`, and print only the date, time, and error code fields and report it in a file `<output>/error_lines.txt`. **[5 Marks]**

sed-a. Use the source tree under `src/`. The tasks below are about precise, one-shot edits across many files.

Some `*.c` and `*.h` files under `src/` contain the line `#include <oldlib.h>`. Create a shell script called `replace.sh` which will replace it with `#include <newlib.h>`, but only in those `*.c` and `*.h` files that also contain a call to `init_adapter()`. Do not modify any other files. If your program creates a backup file, it must have the extension `.bak`. Also create a

file `<output>/patched_files.txt`, listing all files that were actually modified. **[5 marks]**

sed-b. Remove temporary debugging markers: Some source files contain lines beginning with `// TEMPDEBUG:`. Create a shell script named `remove_debug.sh` that removes all such lines from every `*.c` and `*.h` file under `src/`, recursively. Also create `<output>/debug_removed_count.txt`, containing the total number of removed lines. Also your solution should be robust to paths containing spaces wherever that is relevant. **[5 marks]**

2. It is always a good habit to plan ahead. This task will help you to create a courses visualizer in your terminal and find out how many credits are still left to be done.. Strictly use only `sed` and `awk` commands in tasks A and B. You can use loops inside of your `sed` or `awk` commands but what you can't do is use it outside of the `sed/awk` construct i.e. For task A you can use loops inside of your script.

Strictly submit only one file per task and all your logic should be inside that file. i.e. you should not use any helper files.

Resources/Inputs:

1) A CSV file `allCoursesTaken.csv` containing the list of courses taken by a student. The first line of the file contain the headers

`Year, Semester, Code, Name, Credits, Tag, letterGrade`

`Code/Name` refers to the course code/name. `Tag` is the grouping of courses into categories like Departmental Core, Departmental Elective etc.

2) For each tag, `creditsRequirements.csv` gives the credit requirements: ("You need to do 117 credits of Departmental Core courses"). Each tag is also mapped to a color code. You will use it to make the output look more attractive.

3) The mapping of letter grade to a numeric grade point is given in `letterGradeToNumber.csv`

A. Create an `awk` file `viewWithoutColor.awk` which displays the data in `allCoursesTaken.csv` in a formatted way in the terminal. See sample output in `viewWithoutColor.out` to understand what formatting means. Note you need to remove the column `Name` and keep the width of each field 20 columns (use `printf("%20s...)`). Note the number of hyphens for the header is `20*(number_of_fields)`.

Usage:- `awk -f viewWithoutColor.awk \`
`./resources/allCoursesTaken.csv`

From here onwards the output of task A will be called `outputA` **[10 marks]**

- B. Create a script `viewWithColor.sh` which uses `outputA` along with `creditsRequirement.csv` as input. It should use `sed` and `awk` to colorize the output along the lines of the color scheme mentioned in `creditsRequirement.csv`. The file `defineColors.sh` is provided to help you in the task. Look at the intended output in `viewWithColor.out` and `cat` it to understand what to do. Also, look at [this](#) and [this](#) link to understand how the coloring actually works.

Usage :- `./viewWithColor.sh outputA \`
`./resources/creditsRequirements.csv`

Hint:- You may like to use the following `awk` script template. Note that `NR` and `FNR` represent the number of records across all files read so far and the number of records in the current file. So `NR == FNR` is a condition fulfilled while reading the first file.

```
BEGIN {
    # one-time initialization
}
function helper() {...}    # Helper functions you may
need
NR == FNR {
    # build a table which has entries like:
    # display["Basic Sciences"] = BLACK_BACKGROUND CYAN_FONT
    # If you need to exit this and other rules for this input
    # line, use next
}
{
    # Condition fulfilled by the second file,
    # If a line has the tag Basic Sciences, print
    # with color setting display["Basic Sciences"]
}
```

From here onwards output of task B will be named `outputB`

Note:- `cat outputB` will display the text in coloured format. Also use the line
“`source ./resources/defineColors.sh`” in your script to grab hold of
color definitions. **[20 marks]**

- C. Create a script `viewSemester.sh` that takes in 3 arguments. The first is `outputB` and the other two are `semester` and `year`. This filters lines on the basis of `semester` and `year`, sorts the output w.r.t. the `course_code`, and displays it.

Usage:- `./viewSemester.sh outputB Autumn 2018`

The colors should stay the same as in `outputB`.

[5 marks]

D. Create a script `calculateCPI.sh` that takes as input `allCoursesTaken.csv` and `letterGradeToNumber.csv`. For simplicity, assume that all courses are considered in CPI calculation (including Minor, Honor and Additional Learning). Output the CPI correct up to 4 decimal places.

Usage: `./calculateCPI.sh ./resources/allCoursesTaken.csv`
`./resources/letterGradeToNumber.csv` [5 marks]

Restrictions:

- You can assume that `letterGradeToNumber.csv` and `creditsRequirement.csv` are exhaustive i.e. there will not be any grade or tag outside of these.
- The files that you will submit should be only those mentioned in the submission instructions. You should not submit `defineColors.sh` and can assume it to be in `./resources` folder when your scripts are executed.
- You can make as many numbers of intermediate data files during the execution of your scripts but make sure to remove them through your script.
- You can assume that the colors mentioned in `creditsRequirements.csv` will match exactly with one of the colors mentioned in `defineColors.sh`. You should not modify `defineColors.sh` file.

In particular, for this task, you'd want to set your record separators to `"\r\n"`

A plain diff of your output against the expected output should reveal no differences. In particular, please be careful about what order you set the foreground and background colours in! [Total: 40 Marks]

Here's the directory structure for the submission:

```
./<roll-no>
|-- q1
|   |-- failed_login.sh
|   |-- local_filter.sh
|   |-- summary.sh
|   |-- remove_debug.sh
|   `-- replace.sh
`-- q2
    |-- viewCourse.sh
    |-- viewSemester.sh
    |-- viewWithColor.sh
    `-- viewWithoutColor.awk
```

Acknowledgements: Q2 was created by the TAs of the 2019 offering of CS251. I was the instructor for the course.